

Данные и ТВОЙ проект

Анализ текста, защита от ошибок — и твой собственный проект, который ты придумаешь и защитишь сам.

Четверть 4 · 7 уроков + приложения

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 24.** Анализ текста: считаем слова
2. **Урок 25.** Защита от ошибок: try / except
3. **Урок 26.** Повторение: код-битвы
4. **Урок 27.** Выбираем финальный проект
5. **Уроки 28–30.** 🚀 Финальный проект и защита
6. **Приложения.** Шпаргалка, глоссарий, ответы

Анализ текста: считаем слова

ЧЕМУ НАУЧИМСЯ

- Разбивать текст на слова командой **split**
- Считать количество слов и находить самое частое
- Соединять строки, списки и словари для анализа



Текст как данные

Программы умеют не только хранить текст, но и **анализировать** его: сколько слов, какое встречается чаще всего. Так работают поисковики и умные помощники. У тебя уже есть все инструменты — строки, списки, словари. Осталось соединить.



На пальцах. Представь, что рассыпал текст на отдельные слова-карточки. Теперь можно их посчитать, разложить по стопкам, найти самую большую стопку. Именно это делает анализ текста.



Разбираем код

`split` разбивает строку на список слов:

```
analiz.py

text = "кот пёс кот рыба кот пёс"
words = text.split()      # СПИСОК СЛОВ

print("Всего слов:", len(words))

# считаем каждое слово в словарь
count = {}
for word in words:
    if word in count:
        count[word] = count[word] + 1
    else:
        count[word] = 1

print(count)
```

НА ЭКРАНЕ ПОЯВИТСЯ

Всего слов: 6
{ 'кот': 3, 'пёс': 2, 'рыба': 1 }

Цикл идёт по словам. Если слово уже в словаре — прибавляем 1, если нет — начинаем с 1. В итоге словарь показывает, сколько раз встретилось каждое слово. «кот» — 3 раза, чемпион.

ВАЖНО ЗАПОМНИТЬ

`split()` — разбивает строку на список слов по пробелам. Приём «считать в словарь» (есть ключ — прибавь, нет — создай) очень частый: так считают что угодно — слова, голоса, товары.

👉 ЗАДАНИЕ В ПАРЕ

Анализатор предложения

Программа спрашивает предложение и выводит: сколько всего слов и сколько среди них разных (подсказка: разных — это `len(count)`).

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
count[word] = count[word] + 1
# для нового слова — ошибка
```

А ТАК ПРАВИЛЬНО

```
if word in count: ...
else: count[word] = 1
```

Нельзя прибавлять к слову, которого ещё нет в словаре. Сначала проверь, есть ли оно.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Найди самое длинное слово в предложении.
2. Посчитай, сколько раз встречается заданная буква во всём тексте.

★ **Со звёздочкой:** найди и выведи самое частое слово в тексте (не только словарь, а именно чемпиона).

Защита от ошибок: try / except

ЧЕМУ НАУЧИМСЯ

- Ловить ошибки, чтобы программа не падала
- Использовать **try** и **except**
- Делать программы надёжными для любого пользователя



Программа не должна падать

Что будет, если в калькуляторе на вопрос «введи число» пользователь напишет «привет»? Программа **рухнет** с ошибкой. Настоящие программы так себя не ведут — они ловят ошибку и вежливо просят повторить. Это делают слова **try** и **except**.



На пальцах. try / except — это страховка. «Попробуй (try) сделать. Если не вышло (except) — вот запасной план». Как страховочный трос у скалолаза: сорвался — не упал, а повис.



Разбираем код

```
zaschita.py

try:
    age = int(input("Сколько тебе лет? "))
    print("Через год будет:", age + 1)
except:
    print("Это не похоже на число!")
```

ЕСЛИ ВВЕСТИ «ПРИВЕТ»

Это не похоже на число!
(программа не рухнула, а вежливо ответила)

Код внутри **try** — то, что **пробуем** сделать. Если там ошибка — Python не падает, а прыгает в **except** и выполняет запасной план. Пользователь видит понятное сообщение, а не красную простыню ошибки.

ВАЖНО ЗАПОМНИТЬ

try: — рискованный код. **except:** — что делать при ошибке. Особенно полезно с **int(input())**, делением (вдруг на ноль) и файлами (вдруг нет файла).

ЗАДАНИЕ В ПАРЕ

Надёжный калькулятор

Возьмите калькулятор из четверти 1 и защитите его: если ввели не число или делят на ноль — программа не падает, а просит повторить.

Через 10 минут поменяйтесь ролями.

Частые ошибки

ТАК НЕ РАБОТАЕТ

```
try:
age = int(input())
except:
```

А ТАК ПРАВИЛЬНО

```
try:
    age = int(input())
except:
```

Код внутри try и except — с отступом, как в if. Не забудь двоеточия после обоих слов.

ДОМАШНЕЕ ЗАДАНИЕ

1. Защити программу, которая читает файл: если файла нет — сообщи об этом вежливо.
2. Сделай ввод возраста, который повторяет вопрос, пока не введут число (try внутри цикла).

★ **Со звёздочкой:** раздели except на два: отдельно для «не число», отдельно для «деление на ноль».

Повторение: код-битвы и турнир

ЧТО СДЕЛАЕМ

- Повторим всё за год в игровой форме
- Проведём код-битвы у проектора
- Подготовимся к финальному проекту



Урок-турнир

Сегодня без новых тем — **закрепляем** всё, что прошли за год. Но не скучным повторением, а битвами: пары соревнуются, кто быстрее и красивее решит задачу. Это и повторение, и подготовка к финальному проекту.



Правила код-битвы

- 1 Учитель даёт задачу**
Короткую, на 5–7 минут: например, «программа выводит все чётные числа от 1 до 50».
- 2 Пары решают на время**
Кто первый получил рабочий код — поднимает руку.
- 3 Показ у проектора**
Две пары показывают решения. Класс голосует: чей код короче и понятнее.



Задачи для битвы

Проверь себя — сможешь решить каждую? Это карта всего, что ты умеешь:



Числа

Сумма цифр введённого числа



Циклы

Вывести таблицу квадратов 1–10



Строки

Проверить, палиндром ли слово



Списки

Найти среднее и максимум списка



Функции

Функция, бросающая кубик



Словари

Посчитать буквы в слове

ТУРНИР КЛАССА

Кубок программиста

Играем 5–6 раундов. За каждую победу пара получает очко. В конце — чемпионы четверти. Проигравших нет: решил задачу — уже молодец.

Меняйтесь, кто за клавиатурой, каждый раунд.

ДОМАШНЕЕ ЗАДАНИЕ

1. Реши три любые задачи из карты выше, которые показались сложными.
2. Начни думать над темой финального проекта — что тебе интересно?

★ **Со звёздочкой:** придумай свою задачу для код-битвы и её решение.

Выбираем финальный проект

ЧТО СДЕЛАЕМ

- Выберем тему своего проекта
- Составим план: что должна уметь программа
- Распределим работу на три урока



Твой собственный проект

Финал года — проект, который ты придумываешь и делаешь **сам** (в паре). Не по образцу, а свой. Три урока на работу, потом защита перед классом. Это то, что покажет: ты умеешь программировать не по шаблону, а решать свою задачу.



Идеи для проекта

Выбери из этих или придумай своё:



Викторина

Вопросы по любимому предмету, подсчёт очков



Игра-бродилка

Текстовый квест с выбором пути



Копилка

Учёт денег: доходы, расходы, файл



Список дел

Задачи с сохранением в файл



Рисунок turtle

Сложная картина или узор-генератор



Своя игра

Развитие «Змейки» или новая аркада



Как спланировать

1

Опиши идею одним предложением

«Викторина по истории Таджикистана на 10 вопросов со счётом».

2

Раздели на части

Что должна уметь программа? Выпиши 3–5 пунктов.

 ЗАДАНИЕ В ПАРЕ**План проекта**

Выберите тему, опишите идею одним предложением и выпишите, что программа должна уметь. Покажите план учителю до начала кодирования.

Спорьте, обсуждайте — хороший план экономит массу времени.

 ДОМАШНЕЕ ЗАДАНИЕ

1. Доработай план: нарисуй на бумаге, как программа будет выглядеть.
2. Выпиши, какие темы года понадобятся (списки? файлы? функции?).

★ **Со звёздочкой:** продумай одну «фишку», которой удивишь класс на защите.



Твой проект и защита

ФИНАЛЬНАЯ ПРЯМАЯ

- Три урока на создание своего проекта
- Защита перед классом — покажи, что умеешь
- Диплом об окончании курса



План трёх уроков

1 Урок 28 — каркас

Собери основу программы: главное меню или главный цикл. Пусть работает хоть как-то, пусть криво — главное запускается.

2 Урок 29 — начинка

Добавь основные функции: подсчёт очков, сохранение в файл, проверки. Программа делает то, что задумано.

3 Урок 30 — блеск и защита

Доведи до ума: сообщения, красота, «фишка». Подготовь и проведи защиту.



Как пройти защиту

Защита — 3 минуты у проектора. План простой:

1 Покажи в действии

Запусти программу, продемонстрируй, что она делает.

2 Объясни устройство

Расскажи главное: какие темы года использовал, как всё работает.

3 Поделись трудностями

Что было сложно, где застрял, как решил. Это ценят больше всего.

КРИТЕРИИ ОЦЕНКИ ПРОЕКТА

✓ Программа запускается без ошибок · ✓ Решает задачу, которую задумал · ✓ Есть комментарии и понятные имена · ✓ Оба в паре объясняют любую строку · ★ Бонус за «фишку» сверх задания.



ФИНАЛЬНЫЙ ПРОЕКТ ГОДА

Сделай то, чем будешь гордиться

Это твоя главная работа за год. Вложись, добавь свою идею, доведи до конца. Через год ты откроешь этот код и удивишься, как много уже умел.

Помогайте друг другу — но каждый должен понимать весь код.

🎓 ТЫ ПРОШЁЛ ВСЁ КУРС!

За год ты прошёл путь от «что такое команда» до собственных программ и игр. Ты умеешь: переменные, условия, циклы, списки, словари, функции, файлы, графику, обработку ошибок. Это настоящий фундамент программиста.

★ **Что дальше:** продолжай! Придумывай проекты, изучай новое. Python — только начало. Ты уже программист.



Шпаргалка по Python

Всё, что прошли за год, на одной странице. Держи под рукой, когда пишешь код.

ВЫВОД И ВВОД

```
# показать на экране
print("Привет")
# спросить (вернёт текст!)
name = input("Имя: ")
# спросить число
n = int(input("Число: "))
```

ПЕРЕМЕННЫЕ И ЧИСЛА

```
age = 15          # число
name = "Али"      # текст
x + y    x - y    # + -
x * y    x / y    # * /
x % y     # остаток
```

УСЛОВИЯ

```
if age >= 18:
    print("взрослый")
elif age >= 12:
    print("подросток")
else:
    print("ребёнок")
# == сравнить, = положить
```

ЦИКЛЫ

```
# повторить 5 раз
for i in range(5):
    print(i)
# пока условие верно
while x < 10:
    x = x + 1
```

СПИСКИ

```
fruits = ["яблоко", "банан"]
fruits[0]      # первый
fruits[-1]     # последний
fruits.append("x") # добавить
len(fruits)    # сколько
```

СЛОВАРИ

```
ages = {"Али": 15, "Мади": 14}
ages["Али"]      # найти
ages["Сафар"] = 16 # добавить
ages.get("X", "нет")
```

СТРОКИ

```
word = "памир"
len(word)      # длина
word[0]        # буква
word.upper()   # ЗАГЛАВНЫЕ
"a" in word    # есть ли
word.split()   # на слова
```

ФУНКЦИИ

```
def summa(a, b):
    return a + b

x = summa(2, 3)    # x = 5
```

ФАЙЛЫ

```
f = open("f.txt", "w")
f.write("текст\n")
f.close()
# чтение: "r", дописать: "a"
```

ЗАЩИТА ОТ ОШИБОК

```
try:
    n = int(input())
except:
    print("не число!")
```



Словарь терминов

Простыми словами — все понятия, которые встретились за год.

Программа	Список команд для компьютера, которые он выполняет по порядку.
Переменная	Подписанный «ящик» для хранения значения. Кладёшь по имени, достаёшь по имени.
Строка	Текст в кавычках. «привет», «Худжанд» — это строки.
Число	То, с чем можно считать. Бывает целым (15) и дробным (3.14).
Условие	Проверка «если ... иначе ...», по которой программа выбирает действие (if / else).
Цикл	Повторение команд: for (по счёту) или while (пока условие верно).
Список	Много значений в одном ящике, пронумерованных с нуля. В квадратных скобках [].
Словарь	Пары «ключ — значение». Ищешь по ключу-имени, а не по номеру. В фигурных скобках { }.
Функция	Своя команда с именем, созданная через def. Описал раз — вызываешь сколько нужно.
Аргумент	Данные, которые передаёшь функции при вызове: summa(2, 3) — тут 2 и 3.
return	Слово, которым функция отдаёт результат наружу, чтобы его можно было использовать.
Файл	Место, где данные хранятся и после закрытия программы. Открывают через open.
Модуль	Набор готовых команд, который подключают словом import. Например, turtle, random.
Отступ	4 пробела перед командой. Показывает, что она внутри if, цикла или функции.
Ошибка (Error)	Сообщение Python о проблеме в коде. Не провал, а подсказка, что и где поправить.



Ответы к заданиям

Подсказки и готовые решения заданий в паре. Загляни сюда, только если застрял — сначала попробуй сам!

ЧАСТЬ 1 — ПЕРВЫЕ ПРОГРАММЫ

Урок 2 · Магази́нный чек

```
price = int(input("Цена: "))
count = int(input("Количество: "))
print("Итого:", price * count, "сомони")
```

Урок 3 · Больше из двух

```
a = int(input("Первое: "))
b = int(input("Второе: "))
if a > b:
    print(a)
else:
    print(b)
```

Урок 5 · Сумма от 1 до 100

```
total = 0
for i in range(1, 101):
    total = total + i
print(total)    # 5050
```

ЧАСТЬ 2 — СТРУКТУРЫ И ФУНКЦИИ

Урок 9 · Магази́нная корзина

```
prices = [12, 45, 8, 30, 15]
print("Всего:", sum(prices))
print("Дороже всего:", max(prices))
```

Урок 11 · Калькулятор на функциях

```
def summa(a, b):
    return a + b
def raznost(a, b):
    return a - b

x = int(input("a: "))
y = int(input("b: "))
print(summa(x, y), raznost(x, y))
```

ЧАСТЬ 3 — ГРАФИКА И ФАЙЛЫ

Урок 15 · Треугольник

```
import turtle
t = turtle.Turtle()
for i in range(3):
    t.forward(100)
    t.left(120)
turtle.done()
```

Урок 16 · Пятиугольник

```
for i in range(5):
    t.forward(80)
    t.right(72)    # 360 / 5 = 72
```

ЧАСТЬ 4 — ДАННЫЕ

Урок 24 · Анализатор предложения

```
text = input("Предложение: ")
words = text.split()
print("Всего слов:", len(words))
print("Разных:", len(set(words)))
```

Урок 25 · Надёжный калькулятор

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except:
    print("Ошибка: проверь ввод!")
```

💡 Остальные задания решаются по этим образцам. Если не получается — вернись к соответствующему уроку и перечитай раздел «Разбираем код».